



SQLite-Carver-Pro

Guide Forensique Officiel & Manuel de Référence Technique

Édition v1.3.0 — Forensic Analysis Edition

Points Clés Traités dans ce Manuel :

- Algorithmique du Carving SQLite de bas niveau & Optimisations $O(1)$
- Explication mathématique du Seuil de Confiance (0.00 à 1.00)
- Freeblocks, Slack Space, Espace non alloué & Freelist
- Récupération résiliente des chaînes de texte tronquées dans le Slack
- Moteur d'Horodatages Forensiques Multi-Formats (Unix s/ms/ μ s, WebKit, FILETIME, Mac Cocoa, GPS)
- Analyseur de Signatures Fichiers BLOB & Extraction Automatisée (--dump-blobs)
- Détection et Isolation des Tables WAL-Only (★ WAL-ONLY TABLE)
- Filtrage strict des faux positifs Protobuf & Décodage zlib/bplist
- Timeline WAL interactive & Diffs visuels avant/après (avec filtre --table)
- Support Multilingue (Français / Anglais) & Routeur d'export unifié

Auteur : Dam-FOR3K (avec l'assistance technique d'Antigravity / Google DeepMind)

Compatibilité : Windows 64-bit / Linux / macOS (Python 3.11+ & Binaire autonome .exe)

Date du document : Septembre 2026 (Édition v1.3.0)

Table des Matières

1. Avertissement Légal & Obligation de Vérification

2. Introduction & Architecture Globale

3. Fonctionnement Interne de SQLite & Mécanismes de Carving

- 3.1. Anatomie physique d'un fichier SQLite (.db)
- 3.2. Types de pages B-Tree & Encodage des Varints
- 3.3. Les 4 zones de carving forensique (Calcul exact des intervalles de Slack)
- 3.4. Reconstruction des chaînes de débordement (Overflow)
- 3.5. Journal Write-Ahead Log (WAL) & Fusion automatique dans Carve
- 3.6. Décodage automatique des payloads binaires imbriqués (Apple bplist, Protobuf, zlib)
- 3.7. Moteur d'Horodatages Forensiques Multi-Formats (Unix s/ms/µs, WebKit, FILETIME, Cocoa, GPS)
- 3.8. Analyseur de Signatures Fichiers BLOB & Carving Automatique (Images, PDF, ZIP, SQLite)
- 3.9. Détection et Isolation des Tables WAL-Only

4. Comprendre le Seuil de Confiance Forensique (Confidence Threshold)

- 4.1. Pourquoi un seuil de confiance ?
- 4.2. Formule mathématique, Invariants SQLite & Barème des structures imbriquées
- 4.3. Guide d'interprétation des 4 paliers de confiance
- 4.4. Comment choisir le bon seuil selon le scénario d'enquête

5. Guide d'Utilisation de la Ligne de Commande & du .exe

- 5.1. Inspection de la base (`info`)
- 5.2. Carving des données supprimées & Extraction BLOBs (`carve --dump-blobs`)
- 5.3. Recherche forensique récursive (`search`)
- 5.4. Différentiel des transactions (`wal-diff` avec filtre table)
- 5.5. Microscope binaire indépendant avec métadonnées (`decode-blob`)

6. Rapports Forensiques & Formats d'Export

- 6.1. Le Tableau de Bord HTML Interactif à Double Vue (All Evidence & Timeline WAL avec Pagination 60 FPS, Badges d'Horodatage et Sélecteur Bilingue)
- 6.2. Format JSON Indenté (`.json`) & JSON Lines (`.jsonl`)
- 6.3. Format Tabulaire Excel (`.csv`) & Parquet (`.parquet`)

7. API Python pour Pipelines d'Investigation

1. Avertissement Légal & Obligation de Vérification

⚠ Recommandation Méthodologique Fondamentale

Dans toute analyse numérique, **aucune décision critique, opérationnelle ou judiciaire ne doit être prise sur la seule base d'une reconstruction heuristique automatisée**. Les enregistrements récupérés dans les freeblocks, le slack space ou les journaux WAL non validés peuvent avoir été partiellement altérés par des écritures ultérieures. L'analyste doit impérativement corroborer les données extraites, les horodatages décodés et les signatures identifiées par d'autres sources de preuves indépendantes. Le logiciel est fourni sans garantie d'aucune sorte.

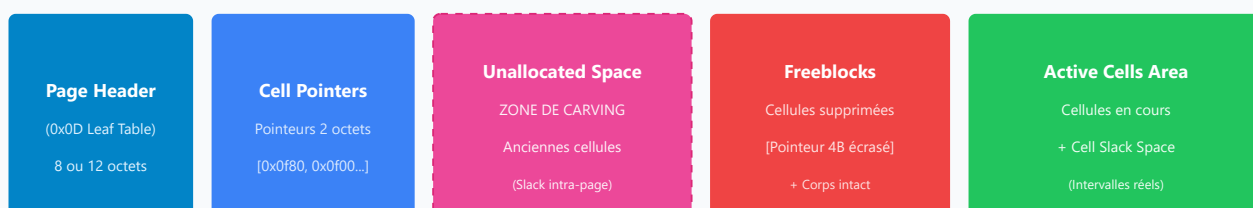
2. Introduction & Architecture Globale

Dans les analyses forensiques, les bases de données SQLite constituent l'un des conteneurs de preuves les plus fréquents (historiques de navigation web, messageries instantanées chiffrées de type Signal, Telegram, WhatsApp, applications mobiles iOS et Android, journaux d'événements système, tokens d'authentification).

Les outils de carving traditionnels se limitent généralement à une extraction de chaînes de caractères brutes (Greedy Text Dump), ignorant la structure relationnelle, les schémas de tables, les types de colonnes, et les structures binaires complexes sérialisées au sein des colonnes BLOB.

SQLite-Carver-Pro v1.3.0 a été conçu comme un moteur forensique complet de niveau système capable de disséquer la structure B-Tree binaire, de retrouver les résidus de cellules dans les zones non allouées et les freeblocks, de recoller les chaînes de débordement multi-pages, de déchiffrer les structures imbriquées (Apple Binary Plist, Google Protocol Buffers, flux compressés zlib/deflate), de décoder automatiquement les horodatages sous de multiples standards chronologiques et d'extraire les fichiers binaires intégrés.

Anatomie Visuelle d'une Page SQLite B-Tree Leaf (4096B)



3. Fonctionnement Interne de SQLite & Mécanismes de Carving

3.1. Anatomie physique d'un fichier SQLite (.db)

Un fichier SQLite est découpé en pages de taille fixe (généralement 4096 octets, ou 512 à 65536 octets). La page 1 contient l'en-tête de base de données de 100 octets.

3.2. Types de pages B-Tree & Encodage des Varints

Les enregistrements SQLite utilisent des **Varints** (entiers à longueur variable de 1 à 9 octets en Big-Endian) et une table de **Serial Types** pour déterminer le type et la taille de chaque colonne :

Serial Type	Type de Donnée & Longueur	Signification Forensique
0	0 octet	Valeur <code>NULL</code> .
1, 2, 3, 4, 5, 6	1, 2, 3, 4, 6, 8 octets	Entiers signés (avec extension de signe pour entiers 24-bit et 48-bit).
7	8 octets	Nombre à virgule flottante IEEE 754 (64-bit float).
8, 9	0 octet	Constante numérique entière 0 ou 1 (optimisation SQLite).
$N \geq 12$ (Pair)	$(N - 12) / 2$ octets	Données binaires brutes BLOB (ex: images, bplist, protobuf).
$N \geq 13$ (Impair)	$(N - 13) / 2$ octets	Chaîne de caractères TEXT encodée en UTF-8.

3.6. Décodage automatique des payloads binaires imbriqués

Les applications modernes encapsulent fréquemment des structures sérialisées au sein des colonnes BLOB :

- **Apple Binary Plist (`bplist00`)** : Décryptage arborescent des dictionnaires, tableaux, chaînes et entiers.
- **Google Protocol Buffers** : Décodeur dynamique de wire format extrayant les paires champ/valeur sans fichier `.proto` préliminaire, avec élimination rigoureuse des faux positifs.
- **Flux compressés Zlib / Deflate** : Décompression automatique et détection de texte clair ou structures sous-jacentes.

3.7. Moteur d'Horodatages Forensiques Multi-Formats (v1.3.0)

Les bases de données applicatives stockent leurs horodatages sous une grande diversité de représentations temporelles, rendant la corrélation chronologique fastidieuse lorsqu'elle est manuelle. **SQLite-Carver-Pro v1.3.0** intègre un moteur heuristique de décodage temporel multi-formats (`sqlite_carver.decoders.timestamps`) capable de reconnaître et convertir instantanément en UTC les formats suivants :

Format Temporel	Époque & Unité	Ordre de Grandeur Typique (2020-2030)	Applications Rencontrées
Unix Epoch (Secondes)	1er Janvier 1970 (secondes)	1.6e9 - 1.9e9 (10 chiffres)	Systèmes Linux, APIs REST, SQLite natif
Unix Epoch (Millisecondes)	1er Janvier 1970 (millisecondes)	1.6e12 - 1.9e12 (13 chiffres)	JavaScript, Android SQLite, Telegram
Unix Epoch (Microsecondes)	1er Janvier 1970 (microsecondes)	1.6e15 - 1.9e15 (16 chiffres)	Bases haute précision, journaux kernel
WebKit / Safari / Chrome	1er Janvier 1601 (microsecondes)	1.3e16 - 1.4e16 (17 chiffres)	Google Chrome (History), Chromium, Safari
Windows FILETIME	1er Janvier 1601 (100 nanosecondes)	1.3e17 - 1.4e17 (18 chiffres)	NTFS, Windows Registry, Skype, Teams
Apple Cocoa Absolute Time	1er Janvier 2001 (secondes)	6.0e8 - 9.0e8 (9 chiffres)	iOS Photos, Messages Apple, Safari macOS
GPS Time	6 Janvier 1980 (secondes)	1.2e9 - 1.5e9 (10 chiffres)	Télémétrie de navigation, géolocalisation

Le moteur utilise un filtrage contextuel intelligent basé sur le nom des colonnes (`time`, `date`, `created`, `timestamp`, `sent`, etc.) et des fenêtres de validité calendaires strictes (1995-2040) pour éliminer tout faux positif.

3.8. Analyseur de Signatures Fichiers BLOB & Carving Automatique (v1.3.0)

Les colonnes de type BLOB contiennent régulièrement des fichiers entiers incrustés (photos de profil, pièces jointes, bases de données de cache). Le module `sqlite_carver.decoders.blobs` identifie automatiquement les nombres magiques et extensions associées :

- **Images** : PNG (`\x89PNG\r\n\x1a\n`), JPEG (`\xff\xd8\xff`), GIF (`GIF87a` , `GIF89a`), WebP (`RIFF...WEBP`), TIFF (`II*\x00` ou `MM*\x00*`).
- **Documents & Archives** : PDF (`%PDF-`), ZIP / APK / DOCX (`PK\x03\x04`), GZIP (`\x1f\x8b`).
- **Bases imbriquées** : SQLite Database Header (`SQLite format 3\x00`).

L'option CLI `--dump-blobs <répertoire>` permet d'extraire automatiquement l'ensemble de ces artefacts sous forme de fichiers prêts à l'analyse externe.

3.9. Détection et Isolation des Tables WAL-Only (v1.3.0)

Dans certaines activités suspectes ou opérations de dissimulation, des tables peuvent être créées et manipulées au sein du journal Write-Ahead Log sans avoir été répercutées dans le fichier de base de données principal (par exemple avant un arrêt brutal ou une transaction non commitée). **SQLite-Carver-Pro v1.3.0** isole ces tables et leur attribue un statut spécifique `[★ WAL-ONLY TABLE]` pour attirer l'attention prioritaire de l'analyste.

4. Comprendre le Seuil de Confiance Forensique (Confidence Threshold)

4.1. Pourquoi un seuil de confiance ?

Lorsqu'une base de données SQLite est corrompue ou lorsqu'on explore l'espace non alloué et les freeblocks, le disque dur contient des milliers d'octets résiduels. Si l'on applique un décodeur sans validation stricte, n'importe quelle séquence aléatoire d'octets pourrait être interprétée à tort comme un enregistrement SQLite valide (faux positif).

Pour garantir une **fiabilité d'analyse**, `sqlite-carver-pro` attribue à chaque enregistrement récupéré un **Score de Confiance compris entre 0.00 et 1.00**.

4.2. Formule Mathématique de Calcul du Score & Barème des Structures Imbriquées

Le score de confiance est calculé dynamiquement selon 5 piliers stricts :

1. Invariant de Longueur de Payload (Physique) :

La formule physique $\text{taille_payload} = \text{taille_header} + \text{somme}(\text{longueurs})$ doit être rigoureusement respectée. Lors de la découverte d'une cellule valide, le pointeur de scan avance exactement de la taille totale de la cellule pour éviter tout chevauchement ou doublon fantôme.

2. Validation des Caractères Imprimables (Sanity Check) :

Pour chaque colonne identifiée comme du `TEXT` (serial type impair ≥ 13), le moteur vérifie que 100% des octets correspondent à des caractères UTF-8 / ASCII valides et imprimables. Si du bruit binaire non décodable est présent, le score chute drastiquement.

3. Appariement Heuristique de Schéma & Score Intrinsèque :

Le moteur compare la séquence de types trouvée avec les schémas découverts dans `sqlite_schema`. Si aucun schéma n'est disponible (ex: table orpheline ou détruite), un score intrinsèque dynamique est calculé en fonction de la cohérence des types et de la lisibilité des textes (annoté explicitement comme *unmapped schema* dans les détails).

4. Pénalité d'En-tête Écrasé (Incomplete Record Penalty) :

Dans les *freeblocks*, SQLite écrase les 4 premiers octets avec son pointeur de liste chaînée. L'outil reconstruit la cellule à partir de l'offset + 4 mais applique une pénalité structurelle (score initial fixé à `0.85`).

5. Barème Calibré des Structures Imbriquées (BLOBs) :

Pour éviter l'inflation artificielle de confiance sur des formats sans en-tête magique :

- **Apple Binary Plist (`bp1list00`)** : `1.00` (Signature magique stricte + parsing certifié).
- **Flux Zlib / Deflate + Texte clair** : `0.95` (Validation CRC/Adler32 + UTF-8 imprimable).
- **Flux Zlib + Protobuf ou Binaire** : `0.85` (Décompression CRC validée).
- **Google Protocol Buffers (Multi-champs)** : `0.85` (Validation des wire types et des numéros de champs).
- **Google Protocol Buffers (Champ unique / Isolé)** : `0.65` (Attribution prudente pour éliminer les faux positifs).

4.3. Guide d'Interprétation des 4 Paliers de Confiance

< 0.50 (Bruit / Rejeté)

0.50 - 0.64 (Slack)

0.65 - 0.84 (Freeblock)

0.85 - 1.00 (Haute Fidélité)

Plage de Score	Niveau de Confiance	Origine Typique & Signification pour l'Enquêteur
0.85 – 1.00	Haute Fidélité (Certitude Maximale)	Cellules actives ou cellules récemment supprimées intactes. Correspondance parfaite avec le schéma de table, types stricts respectés, textes 100% lisibles.
0.65 – 0.84	Bonne Confiance (Lignes Reconstituées)	Cellules issues des Freeblocks dont le pointeur de début a été écrasé, ou lignes avec colonnes NULL/partielles, ou structures Protobuf sans signature.
0.50 – 0.64	Confiance Modérée (Slack & Reliques)	Fragments de cellules trouvés dans le Slack Space ou les Freelist Pages orphelines (tables ayant subi un <code>DROP TABLE</code> dont le schéma a disparu de <code>sqlite_schema</code>).
< 0.50	Bruit Binaire (Rejeté par défaut)	Artefacts fragmentés, octets aléatoires de clés de chiffrement ou résidus de compression. Filtrés automatiquement pour éviter d'inonder le rapport.

5. Guide d'Utilisation de la Ligne de Commande & du .exe

5.1. Inspection de la base (info)

```
# Diagnostic bas niveau, détection des freelists et tables (avec choix de langue)
sqlite-carver-v1.3.0.exe info evidence.db --lang fr
```

Command Prompt – sqlite-carver-v1.3.0.exe info (v1.3.0)

SQLite-Carver-Pro v1.3.0

Moteur Forensique, Carving de Slack, Freelist & Diff WAL

[bold green]Analyse de l'En-tête de Base SQLite[/bold green]: Taille Page 4096B, Version 2 (WAL),
Trouvé 3 Page(s) de Freelist : [6, 7, 5]

[Tables de Schéma Découvertes]: users (Page 2), exfiltration_logs (Page 4), shadow_bank (Page 5)

5.2. Carving des données supprimées & Extraction de Fichiers BLOBs (carve)

Nouveauté v1.3.0 : L'option `--dump-blobs <répertoire>` extrait automatiquement chaque BLOB contenant un fichier identifié (JPEG, PNG, PDF, ZIP, SQLite, etc.) vers le dossier spécifié. Les horodatages numériques sont automatiquement annotés avec leur date UTC décodée.

```
# 1. Carving complet avec extraction automatique des fichiers BLOBs intégrés
sqlite-carver-v1.3.0.exe carve evidence.db --dump-blobs ./artefacts_extraits

# 2. Isoler UNIQUEMENT les enregistrements supprimés (Freeblocks, Freelist, Slack, WAL)
sqlite-carver-v1.3.0.exe carve evidence.db --deleted-only

# 3. Filtrer par seuil de confiance strict (ex: >= 0.80 pour exclure le slack trop fragmenté)
sqlite-carver-v1.3.0.exe carve evidence.db --min-confidence 0.80

# 4. Exporter le tableau de bord web interactif complet avec badges d'horodatages décodés
sqlite-carver-v1.3.0.exe carve evidence.db --export rapport_investigation.html
```

5.3. Recherche forensique récursive (search)

```
# Recherche récursive par défaut incluant base + journal WAL + extraction des blobs découverts
sqlite-carver-v1.3.0.exe search evidence.db "session_token_xyz" --dump-blobs ./blobs_trouves

# Désactiver explicitement l'inclusion du journal WAL
sqlite-carver-v1.3.0.exe search evidence.db "token" --no-include-wal

# Recherche d'un hash ou pattern hexadécimal dans les champs BLOB
sqlite-carver-v1.3.0.exe search evidence.db "deadbeef" --hex
```

5.4. Différentiel des transactions (wal-diff)

```
# Reconstitution complète de l'historique chronologique des transactions (avec flags WAL-Only Table)
sqlite-carver-v1.3.0.exe wal-diff evidence.db evidence.db-wal

# Cibler une table suspecte spécifique
sqlite-carver-v1.3.0.exe wal-diff evidence.db evidence.db-wal --table users --export diff_users.html
```

6. Rapports Forensiques & Formats d'Export

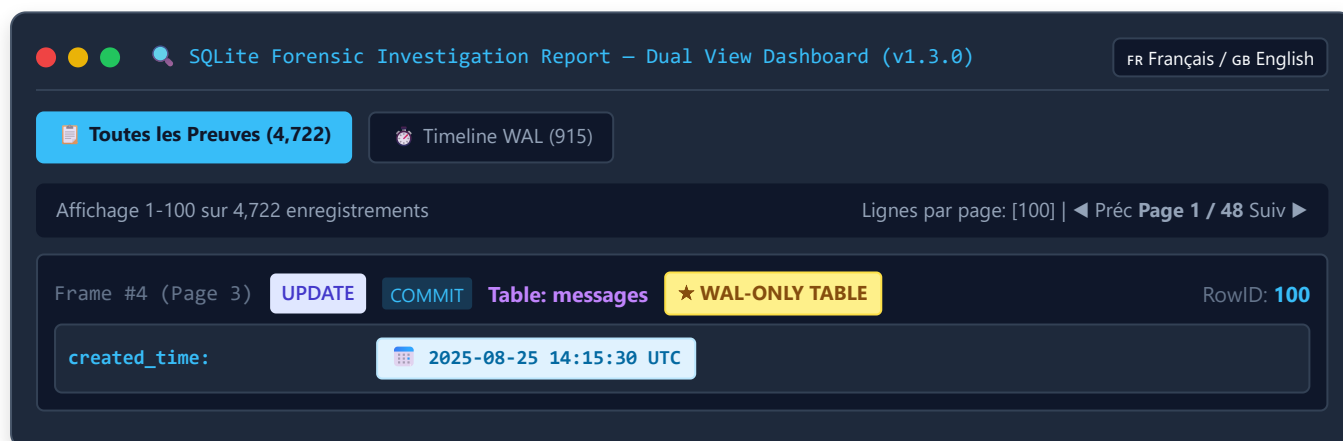
6.1. Le Tableau de Bord HTML Interactif à Double Vue (All Evidence & Timeline WAL avec Pagination 60 FPS, Horodatages Décodés et Sélecteur Bilingue)

L'export HTML génère un **Tableau de Bord Forensique Interactif 100% Autonome (Air-Gapped)** qui s'ouvre directement dans n'importe quel navigateur (Edge, Chrome, Firefox) sans aucune installation requise.

Nouveautés visuelles v1.3.0 :

- **Badges Horodatages** : Chaque entier représentant un timestamp valide est affiché avec un badge calendrier lisible [📅 2026-08-25 14:15:30 UTC] tout en conservant la valeur brute d'origine.
- **Indicateurs Fichiers BLOB** : Les payloads binaires reconnus (PNG, JPEG, PDF, etc.) affichent leur signature détectée et extension suggérée.
- **Badge Tables WAL-Only** : Les tables éphémères créées exclusivement dans le WAL sont mises en évidence par l'étiquette dorée [★ WAL-ONLY TABLE] .

```
sqlite-carver-v1.3.0.exe carve evidence.db --export rapport_investigation.html
```



6.2. Guide des Formats d'Export

Format	Extension	Usage & Public Cible
Rapport HTML	<code>.html</code>	Recommandé pour l'enquêteur. Dashboard interactif bilingue avec recherche temps réel, curseur de confiance, pagination 60 FPS, onglet Timeline WAL, badges d'horodatages décodés et dépliage des BLOBs.
JSON Indenté	<code>.json</code>	Structure claire et aérée avec métadonnées <code>_timestamps</code> enrichies pour analyse sous VS Code / Notepad++.
JSON Lines	<code>.jsonl</code>	Format streaming (une ligne par enregistrement) pour ingestion SIEM (Splunk, ElasticSearch, Cortex XSOAR).
Tableur CSV	<code>.csv</code>	Analyse tabulaire sous Microsoft Excel, LibreOffice Calc, ou importation dans des bases SQL externes.
Apache Parquet	<code>.parquet</code>	Export binaire compressé haute performance pour data science forensique (requêtes Polars, DuckDB, Pandas).

7. API Python pour Pipelines d'Investigation

```
from pathlib import Path
from sqlite_carver import (
    SQLiteCarver, WalDiffEngine, ForensicSearchEngine, export_html,
    TimestampDecoder, detect_file_signature, dump_blob_to_file
)

# 1. Carving programmatique d'une base brute (v1.3.0)
raw_db = Path("evidence.db").read_bytes()
carver = SQLiteCarver(raw_db)

# 2. Extraction avec filtre de confiance (ex: 0.80+)
records = [r for r in carver.carve_all() if r.confidence >= 0.80]
for r in records:
    if r.source != "active":
        print(f"[{r.source.upper()}] Page {r.page_id} | Table: {r.matched_table} | Conf: {r.confidence}")

# 3. Décodage d'horodatages forensiques
ts_decoder = TimestampDecoder()
decoded = ts_decoder.decode(13398765432000000, column_name="visit_time")
if decoded:
    print(f"Horodatage WebKit décodé : {decoded.iso_string} UTC ({decoded.format_name})")

# 4. Carving de BLOBs et détection de signatures de fichiers
for r in records:
    for idx, val in enumerate(r.values):
        if isinstance(val, (bytes, bytearray)):
            sig = detect_file_signature(val)
            if sig:
                print(f"Fichier identifié : {sig.mime_type} ({sig.extension})")
                dump_blob_to_file(val, f"./dump/page_{r.page_id}_col_{idx}")

# 5. Génération automatique du rapport HTML interactif
export_html(records, "rapport_automatise.html")
```

Bonnes Pratiques d'Analyse Recommandées

1. Conservez toujours l'intégrité de la preuve en travaillant exclusivement sur une copie de sauvegarde en lecture seule.
2. Placez toujours les fichiers compagnons `.db-wal` et `.db-shm` dans le même dossier pour l'analyse temporelle.
3. Utilisez le format `.html` pour vos rapports : il permet aux juristes et analystes d'explorer les preuves en toute autonomie.